22360859088
KORAY GARİP

BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Node.JS ile Web Programlama

LAB_1

22360859088
KORAY GARİP

1. Node.js Kurulumu

Öncelikle bilgisayara Node.js kuruldu. Kurulumun doğru yapıldığını kontrol etmek için komut satırında

```
node -v
npm -v
```

komutları çalıştırıldı. Bu komutlar Node.js'in ve npm'in yüklü olup olmadığını ve versiyon bilgilerini gösterir.

2. Tarayıcıdaki JS ile Node.js

Bir web sayfası açılarak geliştirici araçlarından (F12) Console sekmesinde basit JavaScript komutları çalıştırıldı.

```
> window
<  ▶ Window {0: Window, 1: Window, window: Window, self: Window, document: document, name: '', location: Location, …}
> "koray".length
< 5
> 5+3
< 8
```

Benzer işlemler Node ortamında da yapıldı.

```
PS C:\Users\koray> node -v
v22.17.1
PS C:\Users\koray> node
Welcome to Node.js v22.17.1.
Type ".help" for more information.
> 2+9
11
>
> _*2
22
> "KORAY".toLowerCase()
'koray'
>
```

Node ile web ortamında yazılan JavaScript arasındaki farklar karşılaştırıldı. Web için özelleşmiş olan window ve document objelerinin Node ortamında bulunmadığı gözlemlendi. Buna karşılık Node.js ile dosya işlemleri yapılabilirken, temel tarayıcı JavaScript'i ile doğrudan dosya sistemi işlemleri yapılamadığı görüldü.

Sonuç olarak:

JavaScript istemci tarafında çalışabilirken, Node.js JavaScript'i sunucu tarafında çalıştırır.

3. Senkron ve Asenkron Kavramı

Node.js'in en önemli avantajlarından biri asenkron çalışmasıdır.

Senkron

- Kodlar sırayla çalışır.
- Bir işlem bitmeden diğeri başlamaz.
- Bekleme süresi varsa program durur.

Asenkron

- Uzun süren işlemler arka planda çalışır.
- Program beklemeden diğer işlemlere devam eder.
- Performans avantajı sağlar.

Node.js asenkron yapısı sayesinde yüksek performanslıdır.

4. Node.js Örnekleri ve Çalıştırma

Bir h1.js dosyası oluşturuldu ve konsola yazı yazdırıldı.

```
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\koray\Desktop\nodeproject>node h1.js
Hafta bir node giriş

C:\Users\koray\Desktop\nodeproject>
```

Sonraki aşamada senkron şekilde dosya oluşturma ve mesaj verme işlemleri yapıldı.

```
JS h1.js > ...
1  //console.log("Hafta bir node giriş");
2
3  const fs =require("node:fs");
4
5  fs.writeFileSync("notlar.txt","Node.js ' in ilk kodu\n");
6  fs.appendFileSync("notlar.txt","bu not node.js tarafından
   oluşturuldu.")
```

```
notlar.txt
1  Node.js ' in ilk kodu
2  bu not node.js tarafından oluşturuldu.
```

Bir diğer aşamada Node.js ortamında birden fazla JavaScript dosyası oluşturularak dosya içe aktarma işleminin require() ile yapıldığı gözlemlendi. Bir dosya başka bir dosya içinde require("./dosyaAd") şeklinde çağrıldığında, çağrılan dosyanın üst seviyedeki kodlarının örneğin console.log gibi anında çalıştığı görüldü. Ancak başka dosyadaki bir fonksiyonu veya değişkeni kullanabilmek için o dosyada module.exports ile dışa aktarma yapılması gerektiği

öğrenildi. Yani sadece dosyayı çağırmak kodu çalıştırmak için yeterliyken, fonksiyon ya da değişken paylaşımı yapılacaksa module.exports kullanımı zorunludur. Bu sayede projede modüler bir yapı kurularak farklı dosyalardaki fonksiyonlar istenilen yerden çağrılabilir hale getirilmiştir.

```
require("./h1_u3")
console.log("bu ana dosya")
```

```
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\koray\Desktop\nodeproject>node h1_u3_2.js
bu bir yardımcı dosya
bu ana dosya

C:\Users\koray\Desktop\nodeproject>
```

```
JS h1_uyg5.js > <unknown>
const islem = function(s1,s2){
    return s1+s2
}

module.exports=islem
```

```
JS h1_uyg5_2.js > ...
const islem=require("./h1_uyg5")

const sonuc=islem(2,5)
console.log(sonuc)
```

```
JS h1_uyg5.js > islem
const islem = function(){
    return "dönen metin"
}

module.exports=islem
```

```
JS h1_uyg5_2.js > sonuc
const islem=require("./h1_uyg5")

const sonuc=islem()
console.log(sonuc)
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\koray\Desktop\nodeproject>node h1_uyg5_2.js
dönen metin
```

5.NPM

Bu aşamada npm init komutu ile projeye ait package.json dosyası oluşturuldu ve bu dosyanın projenin adı, versiyonu ve bağımlılıklarını tuttuğu gözlemlendi. Daha sonra npmjs.com üzerinden projeleri kolaylaştıran hazır paketlerin bulunduğu incelendi ve bu modüllerin projelerde zaman kazandırdığı, güvenliği artırdığı ve tekrar kod yazma ihtiyacını azalttığı anlaşıldı.

Örnek olarak npm i validator komutu ile yüklenen validator paketi sayesinde email ve URL doğrulama işlemleri hazır fonksiyonlarla yapılabildi; böylece manuel kontrol yazmaya gerek kalmadı.

```
hafta1 > JS h1_uyg6.js > ...
var validator = require('validator');
console.log("email:",
validator.isEmail('a.mail.to.com'),"\nUrl:",

validator.isURL("https://www.npmjs.com/package/validator"))
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\koray\Desktop\nodeproject\hafta1>node h1_uyg6.js
email: false
Url: true

C:\Users\koray\Desktop\nodeproject\hafta1>
```

```
hafta1 > JS h1_uyg6.js > ...
var validator = require('validator');
console.log("email:",
validator.isEmail('foo@bar.com'),"\nUrl:",

validator.isURL("https://www.npmjs.com/package/validator"))
```

```
C:\Windows\System32\cmd.e
Microsoft Windows [Version 10.0.26200.7623]
(c) Microsoft Corporation. Tüm hakları saklıdır.

C:\Users\koray\Desktop\nodeproject\hafta1>node h1_uyg6.js
email: true
Url: true

C:\Users\koray\Desktop\nodeproject\hafta1>
```

Ayrıca npm i chalk ile yüklenen chalk paketi kullanılarak konsol çıktıları renklendirildi ve daha okunabilir hale getirildi.

```
var chalk=require("chalk");
console.log(chalk.green('True'),"\n",chalk.red('False'));
```

```
email: false
Url: true

C:\Users\koray\Desktop\nodeproject\hafta1>node h1_uyg6.js
True
 False

C:\Users\koray\Desktop\nodeproject\hafta1>
```

Bu çalışma ile NPM'in, projeleri daha hızlı, düzenli ve profesyonel şekilde geliştirmeyi sağlayan bir paket yönetim sistemi olduğu öğrenilmiş oldu.

6.NODEMON

Son olarak nodemon aracının global bir modül olarak kurulabileceği öğrenildi. Normalde Node.js dosyaları node app.js komutu ile çalıştırılır ve her değişiklikten sonra uygulamayı manuel olarak durdurup tekrar başlatmak gerekir. Ancak **npm i -g nodemon** komutu ile nodemon global olarak kurulduğunda, sistem genelinde erişilebilir hale gelir ve herhangi bir

proje klasöründe sadece nodemon app.js yazarak kullanılabilir. Nodemon dosyadaki değişiklikleri otomatik olarak takip eder ve her kaydetmede uygulamayı yeniden başlatır. Global modül olması sayesinde her projede tekrar kurulum yapmamızaya gerek olmadan pratik hale getirdi.

```
C:\Users\koray\Desktop\nodeproject\hafta1>npm install -g nodemon

added 28 packages in 10s

5 packages are looking for funding
  run `npm fund` for details

C:\Users\koray\Desktop\nodeproject\hafta1>nodemon h1_uyg6.js
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node h1_uyg6.js`
True
 False
[nodemon] clean exit - waiting for changes before restart
```